scaNMAP

Local Web-Based Zenmap Clone - Documentation

1. Project Overview

scaNMAP is a locally-hosted, web-based graphical interface for the Nmap Security Scanner. Designed as a lightweight clone of Zenmap, it allows users to perform network reconnaissance, view active hosts, and inspect open ports directly from their web browser.

Unlike traditional web applications, this project is designed exclusively for **local execution**, allowing it to leverage elevated system privileges (such as root & administrator access) required for advanced Nmap functionalities like OS detection and SYN stealth scanning.

2. System Architecture

The application follows a simple, monolithic client-server architecture tailored for a localhost environment:

- **Backend:** Python with Flask. Handles API requests, session management, and spawns asynchronous threads for running scans.
- **Scanner Core:** python-nmap module acting as a wrapper around the system's native Nmap binary.
- **Frontend:** Single-page HTML application utilizing Vanilla JavaScript (ES6) and inline CSS for dynamic rendering without relying on heavy frameworks.
- **Database:** SQLite3 for persistent storage of user credentials and historical scan logs.

3. Prerequisites & Installation

3.1. System Requirements

- Python 3.7 or higher
- Nmap installed on the host system system-wide.

3.2. Setup Instructions

1. Ensure Nmap is installed and accessible in your system's PATH.
2. Create a project directory (e.g., mkdir scaNMAP && cd scaNMAP).
3. Create a virtual environment: python -m venv venv
4. Activate the virtual environment.
5. Install dependencies: pip install Flask python-nmap werkzeug
6. Save the backend code as app.py and frontend code as templates/index.html.

4. Usage & Execution

Because advanced scans require raw socket access, the Flask application must be run with elevated privileges.

Important: On Linux/macOS, run using sudo: sudo python app.py. On Windows, open your Command Prompt or PowerShell as an Administrator before executing python app.py.

Once running, open a web browser and navigate to http://127.0.0.1:5000.

5. Features

- **Inline Authentication:** Multi-user support with secure password hashing using Werkzeug.
- **Asynchronous Scanning:** Scans run in a background thread, preventing browser timeouts and keeping the UI responsive.
- **Scan Profiles:** Includes pre-configured Nmap arguments (Stealth, Intense, Ping Sweep, Vulnerability Scripts).
- **Persistent History:** All completed and failed scans are logged to SQLite and can be recalled by clicking on them in the history sidebar.

6. Database Schema

The application initializes a local SQLite database (zenmap_local.db) with two primary tables:

Table: users

| Column | Type | Description |

| :--- | :--- | :--- |

| id | INTEGER | Primary Key (Auto-increment) |

| username | TEXT | Unique user identifier |

| password_hash | TEXT | Securely hashed password |

Table: scans

| Column | Type | Description |

| :--- | :--- | :--- |

| id | TEXT | Primary Key (UUID string) |

| username | TEXT | Foreign Key linking to users table |

| target | TEXT | The IP or hostname scanned |

| arguments | TEXT | The Nmap flags used (e.g., -sS -O) |

| status | TEXT | 'running', 'completed', or 'failed' |

| results | TEXT | JSON string of parsed Nmap output |

7. API Endpoints Reference

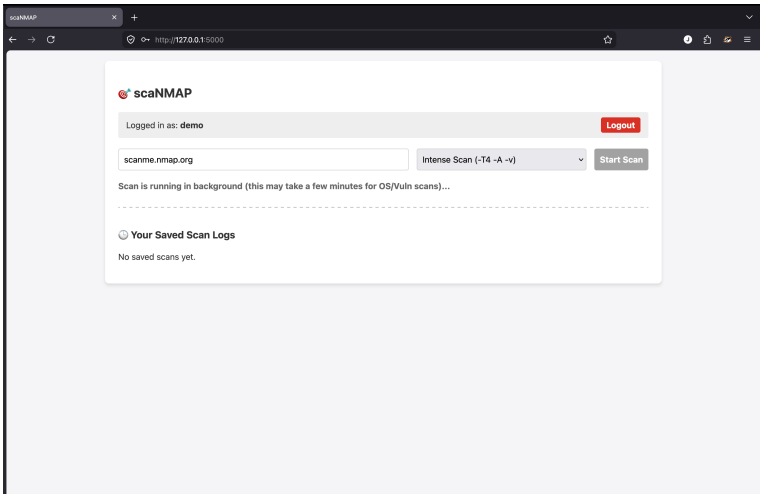
Authentication

- POST /api/auth - Accepts JSON {"username", "password", "action"} where action is 'login' or 'register'.
- POST /api/logout - Clears the current user session.
- GET /api/user - Returns the currently logged-in username.

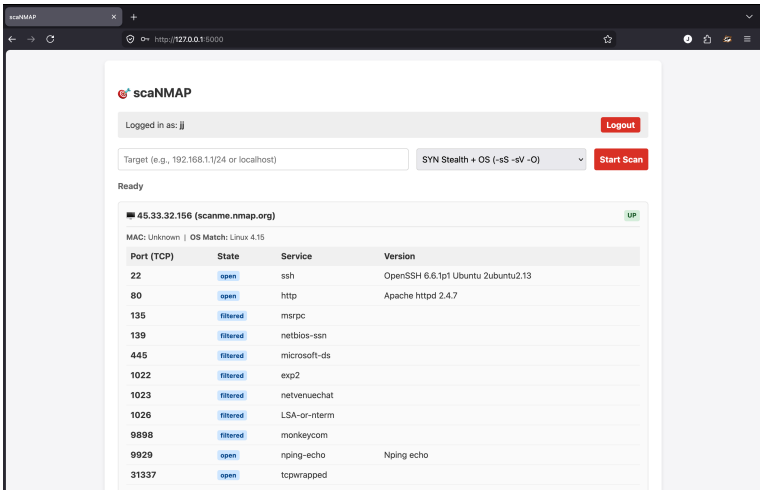
Scanning & History

- POST /api/scan - Starts a new background scan. Requires authentication. Accepts {"target", "type"}. Returns a scan_id.
- GET /api/scan/<scan_id> - Polls the status of an ongoing scan. Returns 'running' or the full results if 'completed'.
- GET /api/history - Fetches all historical scans associated with the logged-in user, ordered by most recent.

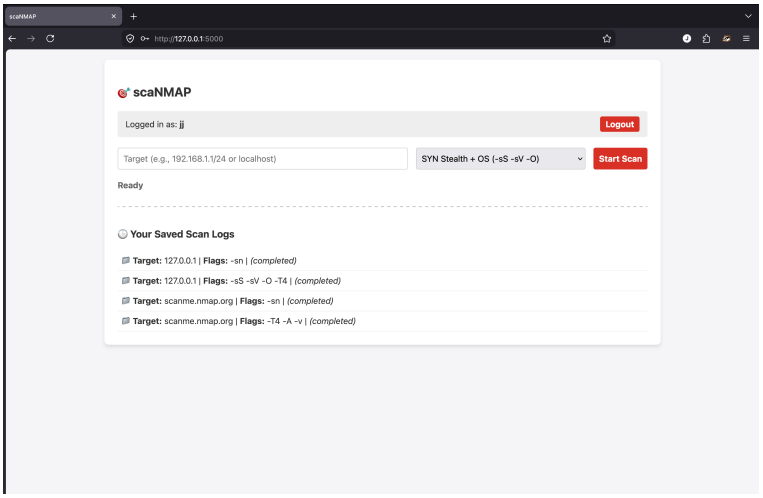
MVP Screenshots



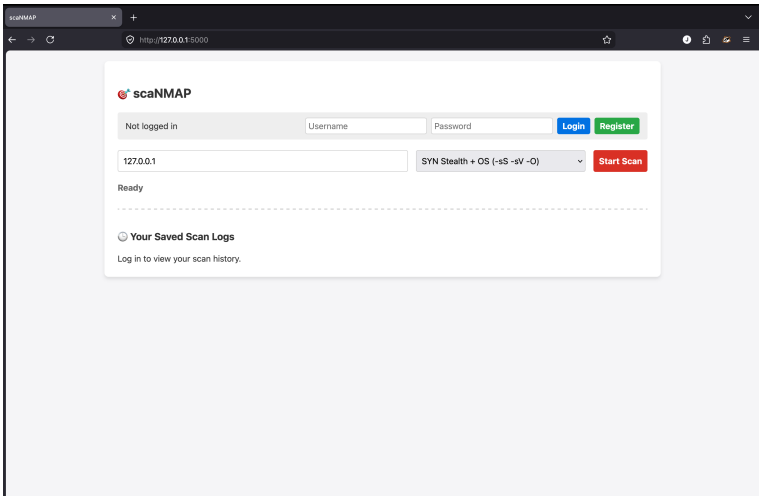
Scan running on input IP
(publicly scannable IP used for testing)



Completed scan results displayed with port numbers, port states, services running and version (if correct scan type chosen).



Past history of scans saved as per user logged in. Selecting any one will show scan
results as shown above.



No scan history is retained after user is logged out.